

1. Student Class

Create a class Student with:

- Properties: name, id, cgpa
- Method: displayInfo()

Task:

- Create 3 student objects and print their details.

2. Bank Account

Create a class BankAccount:

- Properties: accountNumber, balance
- Methods:
 - deposit(amount)
 - withdraw(amount)

Add validation (e.g., cannot withdraw more than balance).

3. Rectangle Area

Create a class Rectangle:

- Properties: length, width
- Method: area()

Create objects and calculate area.

4. Inheritance (Vehicle System)

Create:

- Base class Vehicle with brand and speed
- Derived classes:
 - Car
 - Bike

Add method showDetails() in each class.

5. Method Overriding (Employee Salary)

Create:

- Base class Employee with method calculateSalary()
- Subclasses:
 - FullTimeEmployee
 - PartTimeEmployee

Override salary calculation differently.

6. Abstract Class (Shape System)

Create:

- Abstract class Shape with method area()
- Subclasses:
 - Circle
 - Rectangle

Use polymorphism to call area().

7. Interface Implementation

Create:

- Interface Printable
- Class Report implements Printable

Implement method printData().

8. Library Management System (Mini Project)

Classes:

- Book (title, author, available)
- User
- Library

Features:

- Borrow book
- Return book
- Show available books

9. Constructor & Named Constructor

Create a class Product:

- Properties: name, price
- Default constructor
- Named constructor: Product.discounted()

Apply discount logic.

10. Encapsulation

Create class Person:

- Private variable `_age`
- Getter and setter for age

Add validation (age must be positive).

◆ Challenging Problems (Exam Level)

11. Polymorphism + List

Create a list of different Shape objects and:

- Store them in a single list
- Loop through and calculate total area

12. Multiple Inheritance via Interfaces

Create:

- Flyable, Swimmable
- Class Duck implements both

13. Real-Life Scenario (E-Commerce)

Design system:

- User, Product, Cart, Order

Features:

- Add to cart
- Calculate total price
- Place order

14. Friend-like Behavior (Dart Alternative)

Since Dart doesn't support friend functions like C++, simulate it using:

- Helper class or same library access

Design a secure data-sharing example.

15. Exception Handling in OOP

Create a class ATM:

- Method withdraw(amount)

Throw exception if:

- Insufficient balance
- Invalid amount